

3

Organizing Your React Application

Welcome to a chapter dedicated to unraveling the various strategies to structure a React project. Here, we'll take a step beyond code and delve into the fascinating world of application architecture – an essential aspect of software development that often doesn't receive as much attention in the frontend world as it should.

In this chapter, you will learn about different React project structuring strategies – including the feature-based structure, component-based structure, atomic design structure, and **Model-View-ViewModel (MVVM)** structure – and the unique advantages and potential pitfalls that each approach brings to the table. You'll also be exposed to practical examples of these structures, gain insights into when to use one over another, and explore the trade-offs that come with each decision.

But why should we care about project structure in the first place? A well-structured project can significantly enhance code maintainability, make it easier for new team members to understand the system and improve scalability, and even influence the overall success of the project. In contrast, an inefficient structure can lead to code smells and increased complexity, and it can become a source of technical debt.

By understanding these structuring strategies, you'll be better equipped to make decisions that can have a long-lasting impact on your project's health and success. You'll be able to assess your project's specific needs and constraints and use these strategies as a guiding light, enabling you to create a structure that improves code quality, fosters a productive development environment, and ultimately leads to a successful project.

In this chapter, we will cover the following topics:

- Understanding the problem of a less-structured project
- Understanding the complications of frontend applications
- Exploring common structures in React applications
- Keeping your project structure organized

Technical requirements

A GitHub repository has been created to host all the code we'll discuss in this book. For this chapter, you can find the recommended structure at <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch3>.

Understanding the problem of a less-structured project

The rapid growth of a project can be astonishing, leading to a sense of things spiraling out of control. Initializing a frontend project is typically straightforward, and for small-scale projects, there may be little concern regarding file structure since the number of files to manage is minimal. However, as the project expands, the need for proper file organization becomes apparent:

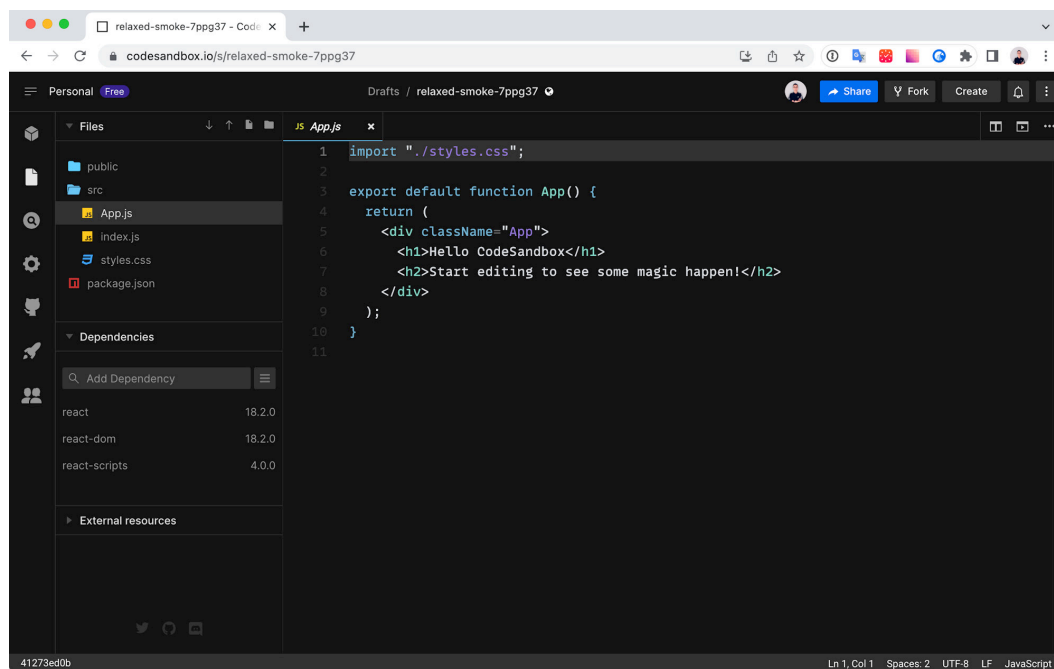


Figure 3.1: A simple project might not need a structure

The problem with having a less-structured project is that it can lead to several challenges and difficulties in managing and maintaining the code base effectively. Some of the key problems that arise from a lack of structure are as follows:

- **Code disorganization:** Without a clear structure, it becomes harder to locate specific code files or components. This can result in wasted time and effort spent searching for relevant code, especially as the project becomes larger.
- **Poor code reusability:** Without a proper structure, it becomes challenging to identify reusable components or functions. This can lead to code duplication and a lack of consistency, making it harder to maintain and update the code base in the long run.
- **Difficulty in collaboration:** When team members are working on a less-structured project, it becomes harder to understand and navigate each other's code. This can lead to communication gaps, slower development, and an increased risk of introducing bugs or conflicts.
- **Scalability issues:** As the project expands and new features are added, the lack of structure can make it challenging to integrate new components seamlessly. This can result in a tangled code base that is difficult to extend or modify, leading to decreased productivity and increased development time.
- **Maintenance complexity:** Without a clear organization, maintaining the code base becomes more complex. Making changes or fixing issues can become a time-consuming task as there may be a lack of consistency in how code is structured or named.

Before proposing a recommended project structure, let's look at the typical components of a modern frontend project. Understanding these components will provide a foundation for designing an effective project structure.

Understanding the complications of frontend applications

In a medium-sized frontend project, you may be surprised by the multitude of components required for its successful implementation. Alongside the core features, numerous other elements contribute to the project's functionality.

The folder structure of a React project provides a glimpse into the various aspects you'll need to manage within a typical React code base:

- **Source code:** This is the heart of your application and contains the JavaScript/TypeScript files that contain your application's logic, HTML files for structure, and style files for appearance. Everything that defines the operation and user interface of your application is found here.
- **Assets:** This category holds all the static files, such as images, videos, and fonts, that are utilized by your application. These files are essential in enhancing the visual experience and interaction of your application, contributing to its overall look and feel.

- **Configuration:** These files hold important parameters that control various aspects of your application. From dependency information in the `package.json` and environment-specific variables to the rules for building your project, these files are crucial for the operation and deployment of your application.
- **Tests:** This category is dedicated to ensuring the correctness and stability of your application. It holds all the unit, integration, and **end-to-end tests** that simulate user behavior, validate interactions, and check the functionality of your application, helping to catch and prevent potential bugs.
- **Documentation:** This is where all the informative documents of your application reside. From the README file providing an overview of the project to the API documentation and the style guide for coding, these documents help maintain consistency, understanding, and ease of use for anyone interacting with the project.
- **Build artifacts:** These are the outputs of the build process, including bundled and optimized JavaScript, CSS, and HTML files ready for deployment, and other temporary or diagnostic files that help debug build issues. They're key to distributing your application to end users.
- **Development tools and configurations:** This is the toolkit that enforces code quality, formatting, and version control and facilitates automated testing and deployment processes. They work in the background, ensuring a smooth, error-free, and efficient development process.

Collectively, these diverse components form the foundation of a typical React code base, highlighting the complexity and breadth of considerations involved in medium-sized frontend projects.

Exploring each feature folder can be an enjoyable experience as it unravels a variety of elements:

- Common components, such as modal dialogs, navigation menus, buttons, and cards
- Dedicated components tailored to specific features, such as `SpecialOffer` for special offers that only display on the menu page or `PayWithApple` for pay with ApplePay
- Style definitions using CSS-in-JS or SCSS/LESS code
- Various types of testing code, including unit tests and browser tests
- Calculation logic encapsulated within utility/helper functions
- Custom Hooks for reusable functionality
- Contexts for security, **internationalization (i18n)**, and other specific needs
- Additional configuration files such as `eslint config`, `jest config`, `webpack settings`, and more

Given the multitude of files, how can we arrange them in a manner that facilitates easy navigation and quick access for modifications? While there is no one-size-fits-all solution, consistently organizing the code base can greatly assist in this endeavor.

Consistency is paramount when it comes to naming and structuring code elements. Regardless of the approach that's chosen, it is crucial to maintain uniformity throughout the project. For example, if you decide to place style files alongside their respective components, it is essential to adhere to this convention across all components in the code base.

Similarly, if a `tests` folder is being used to house test files, ensure to maintain this convention consistently across the entire code base. For instance, other naming patterns such as `__tests__` or `specs` should be avoided to prevent confusion and maintain uniformity.

Having grasped the complexity inherent in medium to large-sized projects, and acknowledging the challenges that a disorganized code base can pose, it's time we explore some tried-and-tested approaches to structuring our code. These strategies aim to simplify the development process and make life easier for developers.

Exploring common structures in React applications

There are many different ways of organizing a large React application. In the following subsections, we will discuss the four most common structures:

- Feature-based structure
- Component-based structure
- Atomic design structure
- MVVM structure

Each structure has its own set of benefits and drawbacks, and the choice depends on the specific requirements and complexity of the project. Sometimes, we might need to mix them in some way so that they fit our project-specific needs.

To explore these different structure methods further, we'll use an online shopping application as an example since it's relatively complicated and you should have some familiarity with that domain already. The application also contains elements such as API calls, routers, and state management.

Feature-based structure

Feature-based structure means the application is organized based on features or modules. Each feature contains its own set of components, views, API calls, and state management, allowing for clear separation and encapsulation of functionality.

With a feature-based architecture in the context of online shopping, you can organize your files and folders as follows:

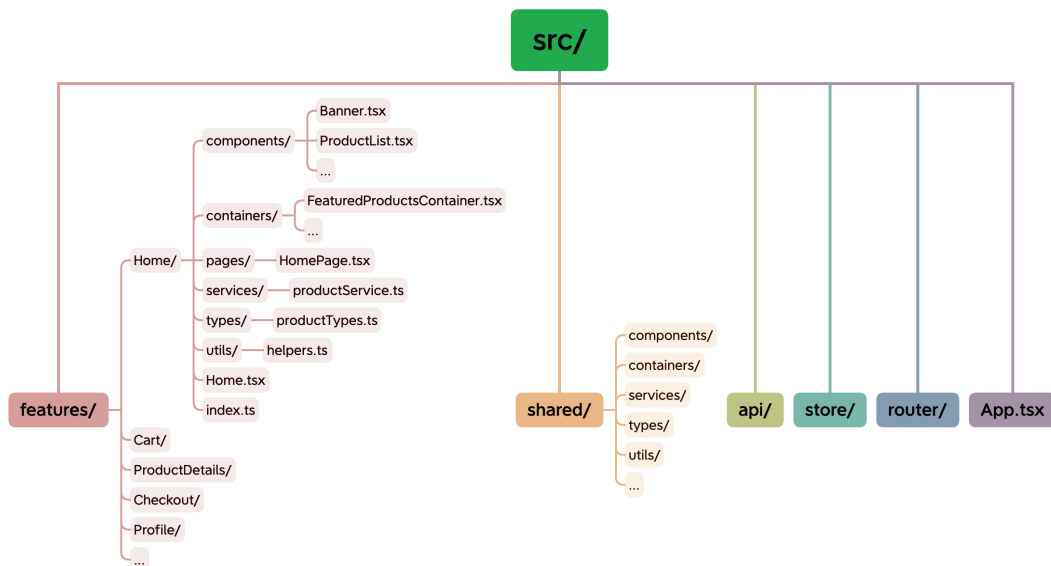


Figure 3.2: Feature-based structure

Let's take a closer look at this structure:

- The `features` directory represents different features of the application, such as `Home`, `Cart`, `ProductDetails`, `Checkout`, `Profile`, and more
- Each feature has a folder containing `components`, `containers`, `pages`, `services`, `types`, and `utils` related to that feature
- The `shared` directory contains reusable `components`, `containers`, `services`, `types`, and `utils` that can be shared across multiple features
- The `api` directory houses modules for making API calls
- The `store` directory contains modules for state management (for example, `Redux`)
- The `router` directory contains the routing configuration and related components
- The `App.tsx` file serves as the entry point of the application

This method has the following benefits:

- **Clear separation of concerns:** Each feature has a folder, making it easier to locate and modify related code
- **Modularity:** Features are self-contained, allowing for easier testing, maintenance, and reuse
- **Scalability:** New features can be added without directly impacting existing code
- **Team collaboration:** Developers can work on different features concurrently with minimal conflicts

However, it has the following drawback:

- **Potential duplication:** Features may share similar components or logic, leading to some duplication. Careful planning and refactoring can help mitigate this.

Component-based structure

Component-based structure means the application is organized around reusable components. Components are categorized based on their functionality and can be composed together to build larger views.

With a component-based architecture in the context of online shopping, you can organize your files and folders as follows:

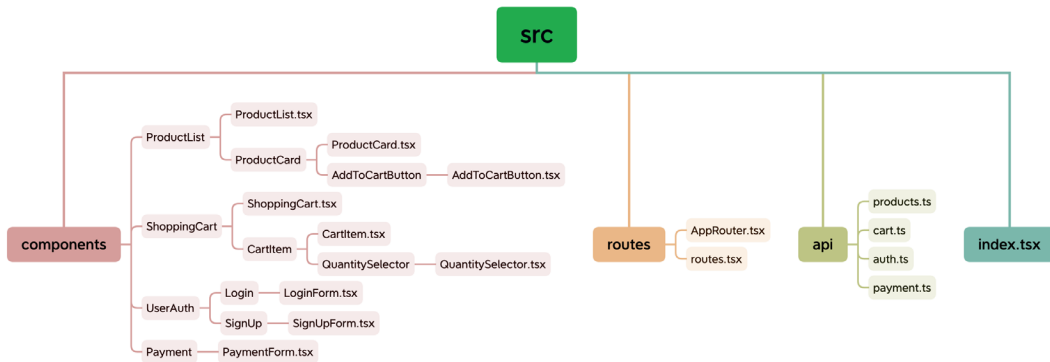


Figure 3.3: Component-based structure

Let's take a closer look at this structure:

- The `components` folder contains individual components related to various features of the online shopping application. Each component is organized into a folder, which may contain child components as necessary.
- The `routes` folder handles frontend routing in the application. It includes the main `AppRouter.tsx` file, which configures the routing logic, and the `routes.tsx` file, which defines the individual routes and their corresponding components.
- The `api` folder contains separate files for different API domains or functionalities. These files, such as `products.ts`, `cart.ts`, `auth.ts`, and `payment.ts`, handle the API calls related to their respective domains.
- The example also assumes the use of a state management library such as Redux or React Context API for managing the global application state.

This method has the following benefits:

- **Modularity:** The component-based structure promotes modularity by organizing components into separate files and folders based on their functionality. This enhances code maintainability and reusability.
- **Separation of concerns:** Each component focuses on its specific functionality, leading to clearer code and easier debugging. The separation of concerns improves code readability and maintainability.
- **Code reusability:** With components organized in a modular structure, it becomes easier to reuse components across the application or in future projects, leading to improved development efficiency.

However, it has the following drawbacks:

- **Project complexity:** As the project grows, maintaining a complex component structure can become challenging. It requires careful planning and adherence to best practices to avoid component sprawl and keep the structure manageable.
- **Learning curve:** The initial learning curve for component-based development and TypeScript may be steeper for developers who are new to these concepts. However, the benefits that are gained in terms of code organization and maintainability outweigh the initial learning.
- **Potential duplication:** Within the `components` folder, you may find smaller elements that are identical or closely resemble those in other component folders. The more you break down these components into finer parts, the greater the likelihood of identifying components that can be reused. When such reusable components emerge, it's a good practice to place them in a "shared" folder, like what is shown in a feature-based structure.

Atomic design structure

Atomic design is a methodology for designing and organizing user interfaces. It emphasizes the construction of user interfaces by breaking them down into small, reusable components called atoms, which are combined to form molecules, organisms, templates, and pages.

The key idea behind atomic design is to create a systematic approach to building UI components that encourages reusability, scalability, and maintainability. It provides a clear structure for organizing and naming components, making it easier to understand and navigate the UI code base.

Here's how the atomic design methodology categorizes UI components:

- **Atoms:** Atoms are the smallest building blocks of a UI and represent individual elements such as buttons, inputs, icons, or labels. They are typically simple and self-contained.
- **Molecules:** Molecules are combinations of atoms and represent more complex UI components. They encapsulate a group of atoms working together to form a functional unit, such as a form field or a navigation bar.

- **Organisms:** Organisms are larger components that combine molecules and/or atoms to create more significant sections of a UI. They represent distinct sections of a user interface, such as a header, sidebar, or card component.
- **Templates:** Templates provide a layout structure for arranging organisms and/or molecules. They define the overall skeleton of a page or a specific section of a UI.
- **Pages:** Pages represent complete user interface screens that are composed of templates, organisms, molecules, and atoms. They represent the final output visible to the user.

With an atomic design architecture in the context of online shopping, you can organize your files and folders as follows:

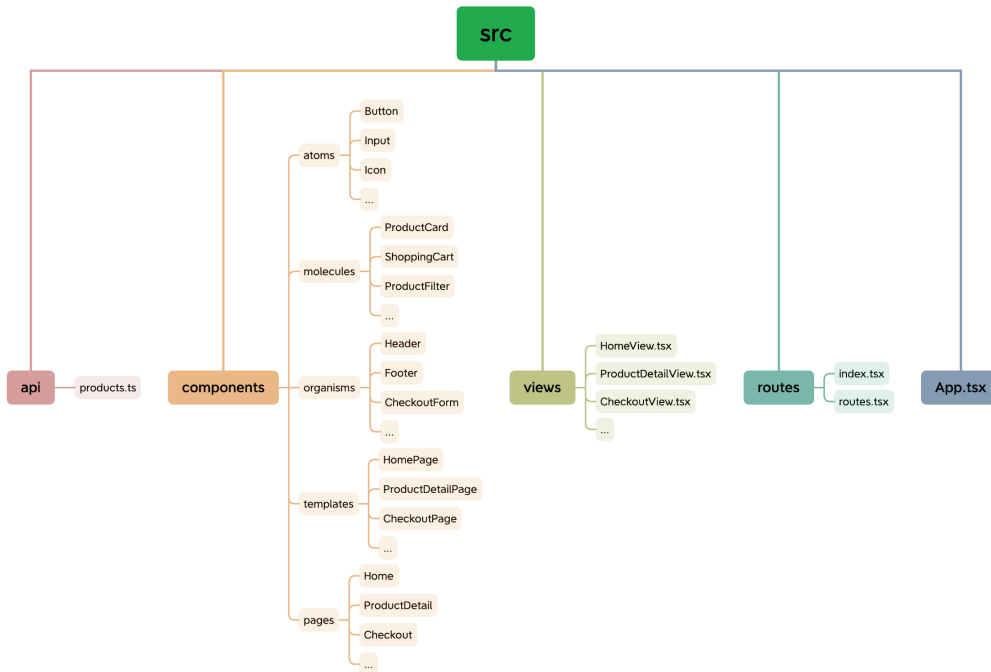


Figure 3.4: Atomic design structure

Let's take a closer look at this structure:

- The `atoms`, `molecules`, `organisms`, `templates`, and `pages` directories represent the different levels of component composition and abstraction
- The `api` directory contains the API-related files for making API calls
- The `views` directory contains the individual views that render the components
- The `routes` directory handles the routing configuration

This method has the following benefits:

- **Reusability:** Components can be easily reused across the application, promoting code efficiency
- **Consistency:** The structure encourages a consistent design language and UI pattern
- **Scalability:** The modular approach allows for easy scaling and the addition of new components
- **Maintainability:** Components are organized in a logical hierarchy, making them easier to locate and update
- **Collaboration:** The atomic design structure facilitates collaboration between designers and developers as it provides a common language for discussing UI components

However, it has the following drawbacks:

- **Learning curve:** It may require some initial learning and adaptation to understand and implement the atomic design principles effectively
- **Complexity:** As the application grows, managing a large number of components and their relationships can become challenging
- **Overengineering:** It's important to strike a balance between component reusability and overengineering as excessive abstraction can introduce unnecessary complexity

The MVVM structure

The **MVVM structure** is a software architectural pattern that's primarily used in building user interfaces:

- The *Model* represents the actual data and/or information we are dealing with. This could be a database, a file, a web service, or even a simple object.
- The *View* is what the user sees and interacts with. It's the user interface that presents the Model to the user.
- The *ViewModel* is where most of the logic resides in this pattern. It is an abstraction of the View that exposes public properties and commands, bridging the gap between the View and the Model, and processes the data from the Model into a format that is easy for the View to handle. It can perform operations on the data and decide how it should be presented to the View.

To structure a React application with the MVVM architecture in the context of online shopping, you can organize your files and folders as follows:

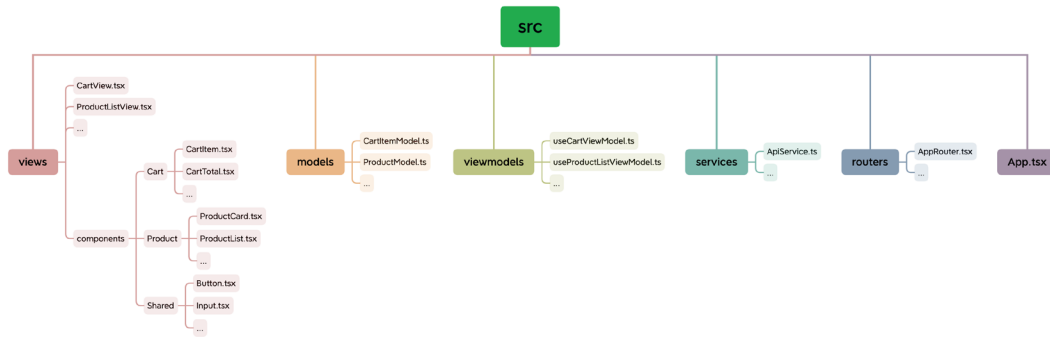


Figure 3.5: The MVVM structure

Let's take a closer look at this structure:

- The `components` directory contains reusable UI components, organized by their respective features
- The `models` directory includes the data models or entities representing the application's domain objects, such as `CartItemModel` and `ProductModel`
- The `viewmodels` directory holds the Hooks responsible for managing the state, logic, and interactions of the views
- The `services` directory contains modules for handling API calls and other external services
- The `views` directory includes the view components that display the UI based on the `ViewModel` state
- The `routers` directory houses the routing configuration and components
- The `App.tsx` file serves as the entry point of the application

This method has the following benefits:

- **Separation of concerns:** The `ViewModel` separates the business logic from the UI components, promoting cleaner and more maintainable code
- **Testability:** The `ViewModel` can be easily unit tested without the need for the actual UI components
- **Reusability:** Components, models, and services can be reused across different features and views
- **Scalability:** New features and views can be added while reusing existing `ViewModel` and service modules

However, it has the following drawbacks:

- **Complexity:** Implementing the MVVM pattern may introduce additional layers of abstraction and complexity to the application, especially for smaller projects
- **Learning curve:** Developers need to understand the concepts and principles of MVVM to effectively structure and manage the application

Now that we have explored these four popular structures, let's delve into the continuous evolution of our application's structure. This ongoing process ensures that the structure remains beneficial for developers in terms of easy navigation, seamless addition of new features, and the ability to maintain scalability over time.

Keeping your project structure organized

The feature-based structure is always a good starting point. As the project expands and patterns of duplication start to emerge, an additional layer can be introduced to eliminate redundancy.

For instance, let's use the online shopping application again. It contains various pages:

- Home page
- Log in/sign up
- Store address search
- Product list
- Shopping cart
- Order details
- Payment
- Profile
- Coupon

In the initial stages, organizing pages based on their features is a common approach. We can create a folder for each feature and put all the related components, styles, and tests inside that folder.

Implementing the initial structure

The initial folder structure in the `src` directory is quite straightforward and follows a feature-based approach, with each page having its own folder:

```
├── Address
│   ├── AddressList
│   └── Store
├── Home
```

```
|— Login
|— Order
|— Payment
|— Product
|— Profile
|   └─ Coupon
└─ SignUp
```

However, as the project evolves, you may encounter duplication of components or functionalities across different pages. To address this, it becomes necessary to introduce an additional layer of abstraction.

For example, if both the `Login` and `Order` pages require a `Button` component, it would be impractical to have separate implementations of the button on each page. Instead, you can extract the `Button` component into a separate layer, such as a `components` or `shared` folder. This way, `Button` can be reused across multiple pages without duplication.

Adding an extra layer to remove duplicates

By adding this extra layer, you promote reusability and maintainability in your code base. It helps in eliminating redundancy, streamlining development efforts, and ensuring consistency throughout the application. As the project expands, this modular approach allows for easy management and scalability, making it easier to add new features or make changes without impacting the entire code base.

So, you can create a `components` folder for all the reusable components and a `pages` folder for all the feature pages, like so:

```
|— components
|   └─ Accordion
|   └─ GenericCard
|   └─ Modal
|   └─ Offer
|       └─ SpecialOffer
|   └─ StackView
└─ pages
    └─ Address
        └─ AddressList
        └─ Store
    └─ Home
    └─ Login
    └─ Order
    └─ Payment
    └─ Product
    └─ Profile
        └─ Coupon
    └─ SignUp
```

As the project expands, it becomes necessary to create a separate components folder to house reusable components that are shared across different pages. In this structure, each component is organized within its respective folder, promoting modularity and code reuse. Additionally, you can introduce nested folders to represent component hierarchies, such as the `Offer` folder, which contains a specific component called `SpecialOffer`.

Alongside the `components` folder, you may need folders for other essential elements. The `pages` folder contains feature-specific pages, while the `hooks` folder houses React Hooks that provide reusable logic and functionality. The `context` folder is used for managing the global state and provides different contexts that can be shared throughout the application.

It's important to note that not all components need to be moved to the `components` folder. Only components that exhibit duplication across different pages should be lifted to the shared folder, ensuring that you maintain a balance between modularity and unnecessary complexity.

This file structure allows for better organization, code reuse, and scalability as the project grows. It promotes maintainability by reducing redundancy and ensuring consistency across the application. Additionally, having a separate folder for Hooks and contexts helps centralize related code and makes it easier to manage and maintain global state and reusable logic.

Naming files

In an individual component, there are different approaches to naming files, and each approach has its advantages and considerations. Let's explore two approaches.

Naming files with `index.tsx` and explicit component names

In this approach, each file within the component folder has an explicit name that corresponds to the component it represents:

```
components/Button
├─ Button.test.tsx
├─ Button.tsx
├─ index.tsx
└─ style.css
```

The `index.tsx` file serves as the default export file, allowing you to import the component from the folder directly. `Button.tsx` is the JSX for the component, and `Button.test.tsx` is the corresponding test file, while `style.css` defines CSS styles.

This approach promotes clear and self-descriptive filenames, making it easier to understand the purpose and content of each file. However, it can result in a long list of index files when browsing or searching for components in an editor or file explorer.

Naming files with kebab case

In this approach, the files within the `components` folder are named using **kebab case**, a naming convention where words are lowercase and separated by hyphens. If there is only one word, simply use lowercase – this follows a consistent convention that’s used in the JavaScript community:

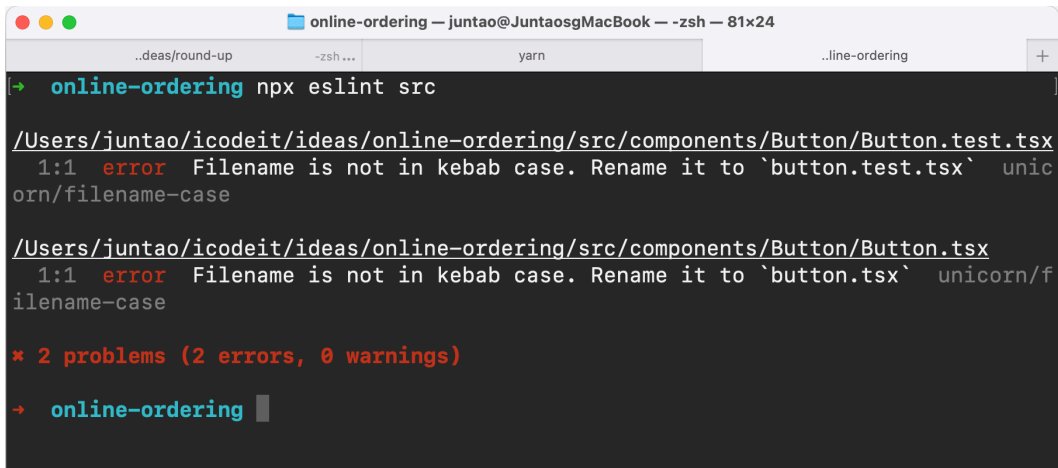
```
components
├─ button.test.tsx
├─ button.tsx
├─ index.tsx
└─ style.css
```

The filename of the component is explicitly named using kebab case (for example, `button.tsx`) to match the component’s name.

This approach maintains consistency with the convention of kebab case filenames and promotes a unified naming structure throughout the project. However, it may require specifying the filename explicitly when importing the component.

Both approaches have their merits, and the choice depends on personal preference and the project’s requirements or team conventions. It is crucial to establish and maintain consistency within the project to enhance collaboration and understanding among team members.

Either way, you can use ESLint and FolderLint to ensure your team has the same naming standard for files and folders. For example, the following screenshot shows that filenames should be in the kebab case and suggests changing `Button.tsx` into `button.tsx`:



```
online-ordering — juntao@JuntaosgMacBook — zsh — 81x24
..deas/round-up  ~zsh ...  yarn  ..line-ordering  +
→ online-ordering npx eslint src

/Users/juntao/icodeit/ideas/online-ordering/src/components/Button/Button.test.tsx
  1:1  error  Filename is not in kebab case. Rename it to `button.test.tsx`  unicorn/filename-case

/Users/juntao/icodeit/ideas/online-ordering/src/components/Button/Button.tsx
  1:1  error  Filename is not in kebab case. Rename it to `button.tsx`      unicorn/filename-case

✖ 2 problems (2 errors, 0 warnings)
→ online-ordering
```

Figure 3.6: ESLint checks

Exploring a more customized structure

As your application grows and different types of abstractions are added, it becomes necessary to organize the project structure accordingly. Relying solely on any previously discussed structure may not be ideal for your specific scenario. It's often necessary to customize the structure so that it aligns well with your project's needs. Remember, the primary goal of establishing a project structure is to simplify and streamline the development process for developers.

Starting with the feature-based structure, we need to adjust our current folder structure to the following folder structure to reflect this evolution:

- `api`: This folder represents the module or directory for managing API-related code, including functions for making network requests, handling responses, and interacting with the backend services.
- `components`: This folder contains reusable UI components that can be used across different pages or features of the application. It includes components such as `Accordion`, `Button`, `GenericCard`, `Modal`, `Offer`, and `StackView`. These components can be organized into subfolders based on their functionality or purpose.
- `context`: This folder represents the module or directory for managing React context, which allows for global state management and data sharing across components.
- `hooks`: This folder contains custom React Hooks that encapsulate reusable logic and behavior. These Hooks can be shared across different parts of the application.
- `mocks`: This folder holds mock data or mock implementations for testing purposes. It includes subfolders for `graphql` and `rest`, which represent mocks for GraphQL and REST APIs, respectively.
- `pages`: This folder represents the different pages or features of the application. Each page or feature has a folder. The folders that are included are `Address`, `Home`, `Login`, `Order`, `Payment`, `Product`, `Profile`, `SignUp`, as well as their respective subfolders. The subfolders may contain additional components, Hooks, or context related to that specific page or feature.

By structuring the project in this way, you can achieve a modular and organized code base that facilitates code reuse, separation of concerns, and scalability. Each directory represents a specific aspect of the application, making it easier to locate and manage code related to that particular functionality.

You can see this in the following figure, where we are returning to our shopping example:

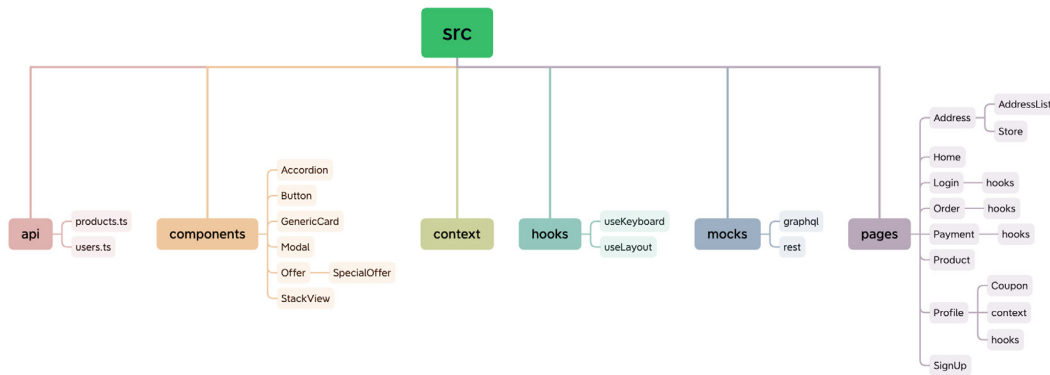


Figure 3.7: A mixed structure for an online shopping application

While this structure provides a solid foundation, it's important to adapt it based on the specific needs and scale of your project. Regularly reviewing and refactoring the structure can help maintain its effectiveness and accommodate future changes.

As the application becomes larger, it may be beneficial to extract the `components` folder into a shared library that can be used across multiple projects or as an internal design system. This approach promotes code reuse, consistency, and maintainability. The shared library can be hosted on an internal registry or published to *npmjs Registry* (<https://www.npmjs.com/>) for easy distribution and consumption.

Also, as your application evolves and new features are introduced, the existing structure may no longer fully meet your requirements. In such cases, it can be beneficial to incorporate architectural patterns such as MVVM, which follows a layered approach. This allows for better separation of concerns and facilitates the management of complex features and state in a more organized manner. We'll have an in-depth discussion about using layered architecture in *Chapter 11*.

Summary

In this chapter, we explored the challenges that arise when managing a large React application and the importance of establishing a solid project structure. We discussed various styles of structuring a React application, including feature-based, component-based, MVVM, and atomic design. Each approach offers its benefits and considerations, allowing developers to choose the most suitable structure for their specific project requirements.

Additionally, we proposed a continuously evolving approach to shaping the folder structure as the project grows. Starting with a simple initial structure, we emphasized the need to adapt and introduce new layers and abstractions to reduce duplication and maintain code organization. By continuously refining the structure and adhering to consistent conventions, developers can navigate, add new features, and maintain scalability effectively.

Throughout this chapter, we highlighted the importance of keeping the project structure flexible and evolving to meet the changing needs of the application. By staying proactive in shaping the folder structure, developers can mitigate the challenges of managing a large React application and ensure maintainability and scalability in the long run.

In the forthcoming chapter, we will delve into prevalent design patterns and strategies for component implementation. These techniques will allow us to craft code that is amenable to feature additions, intuitive to comprehend, and requires less maintenance effort.

4

Designing Your React Components

Welcome to this pivotal chapter on mastering React component design. In this chapter, we'll embark on an enriching journey to recognize and eradicate common anti-patterns in designing React components, including issues such as large monolithic components, prop drilling, and other prevalent pitfalls that often perplex developers and hamper the maintainability and scalability of React applications.

First, we'll introduce the single responsibility principle. In the realm of React, this guides us to ensure that each component has one specific purpose. Adhering to this principle makes components easier to understand, test, and maintain, all while making your code more readable and manageable.

Next, we'll explore the don't repeat yourself principle. One of the core tenets of effective programming, this encourages developers to minimize repetition and promote reuse. In the context of React, this principle can be the key to unlocking a more streamlined, efficient, and maintainable code base.

Finally, we'll delve into the component composition principle. Composition allows us to build complex UIs by combining simpler, reusable components. In React, composition is favored over inheritance, leading to more flexible and easier-to-manage components.

Throughout this chapter, we'll take a deep dive into each of these principles, providing real-world examples and practical applications. By doing so, we aim to guide you in crafting more efficient components, bolstering your understanding of React's potential, and enhancing your problem-solving skills in this powerful library.

So, in this chapter, we will cover the following topics:

- Exploring the single responsibility principle
- Learning about the don't repeat yourself principle
- Using composition
- Combining component design principles

Technical requirements

A GitHub repository has been created to host all the code we discuss in the book. For this chapter, you can find the recommended structure under <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch4>.

Exploring the single responsibility principle

The **single responsibility principle (SRP)** is one of the fundamental concepts in software engineering, asserting that a function, class, or, in the context of React, a component, should have only one reason to change. In other words, each component should ideally handle a single task or functionality. Following this principle can make your code more readable, maintainable, and easier to test and debug.

Let's illustrate this with an example. Suppose you initially have a `BlogPost` component that fetches blog post data, displays the post, and handles the user liking the post, all in one component:

```
import React, { useState, useEffect } from "react";

import fetchPostById from "../fetchPostById";

interface PostType {
  id: string;
  title: string;
  summary: string;
}

const BlogPost = ({ id }: { id: string }) => {
  const [post, setPost] = useState<PostType>(EmptyBlogPost);
  const [isLiked, setIsLiked] = useState(false);

  useEffect(() => {
    fetchPostById(id).then((post) => setPost(post));
  }, [id]);

  const handleClick = () => {
    setIsLiked(!isLiked);
  };

  return (
    <div>
      <h2>{post.title}</h2>
      <p>{post.summary}</p>
      <button onClick={handleClick}>
        {isLiked ? "Unlike" : "Like"}
      </button>
    </div>
  );
};
```

```

    </div>
  );
};

export default BlogPost;

```

The code defines a functional component called `BlogPost` that takes an `id` prop of the `string` type. Inside the component, there are two state variables defined using the `useState` Hook: `post` and `isLiked`. The `post` state represents the blog post data, initialized with a default value of an empty blog post. The `isLiked` state represents whether the post is liked or not, initialized as `false`.

Following this, we need to manage the side effect (sending network requests) in the `useEffect` Hook. It is used to fetch the blog post data from the server based on the provided `id` prop. It triggers the `fetch` operation whenever the `id` prop changes. Once the data is fetched, the `post` state is updated with the retrieved post using the `setPost` function.

Inside the `useEffect` Hook call, for the network request, there is a function called `fetchPostById`. The function is simply a `fetch` call to a remote API endpoint. We can assume the function is implemented with something like the following code snippet:

```

const fetchPostById = (id: string) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve({}), 2000);
  })
};

```

The component renders the title and summary of the blog post from the `post` state. It also renders a button that toggles the `isLiked` state when clicked, displaying **Like** or **Unlike** based on the current value of `isLiked`.

While this code works, it violates the SRP. It's doing three separate things: fetching data, displaying the blog post, and handling the like functionality. Instead, let's refactor it into smaller, single-responsibility components:

```

const useFetchPost = (id: string): PostType => {
  const [post, setPost] = useState<PostType>(EmptyBlogPost);

  useEffect(() => {
    fetchPostById(id).then((post) => setPost(post));
  }, [id]);

  return post;
};

const LikeButton: React.FC = () => {

```

```
const [isLiked, setIsLiked] = useState(false);

const handleClick = () => {
  setIsLiked(!isLiked);
};

return <button onClick={handleClick}>
  {isLiked ? "Unlike" : "Like"}
</button>;
};

const BlogPost = ({ id }: { id: string }) => {
  const post = useFetchPost(id);

  return (
    <div>
      <h2>{post.title}</h2>
      <p>{post.summary}</p>
      <LikeButton />
    </div>
  );
};
```

Here, we've refactored `BlogPost` into smaller, single-responsibility components:

- `useFetchPost` is a custom Hook responsible for fetching the blog post data
- `LikeButton` is a component responsible for handling the like functionality
- `BlogPost` is now just responsible for rendering the blog post content and `LikeButton`

Each part has a single responsibility and could be tested and maintained independently, leading to a more manageable code base.

Note

In a real-world application, clicking a **Like** button may send an API call to update the database; for example, it could be another `fetch` request that is sent to an endpoint such as `https://post.service/post/<id>` to change the `isLiked` status.

So, in the first section, we explored the SRP. This principle encourages each component to take charge of a single piece of functionality, making our code more maintainable and understandable. Here, we applied this principle to break down large, monolithic components into smaller, more manageable pieces.

As we advance further into our design journey, our next section leads us to a principle that intertwines closely with the philosophy of SRP – the don't repeat yourself principle.

Don't repeat yourself

The **don't repeat yourself** (DRY) principle is a fundamental concept in software development that aims to reduce repetition within the code. Following this principle leads to better maintainability, readability, and testability, and can prevent bugs that occur due to the duplication of logic.

Let's say that you have a shopping website, and you want to display a list of products and the user's cart side by side, something like this:

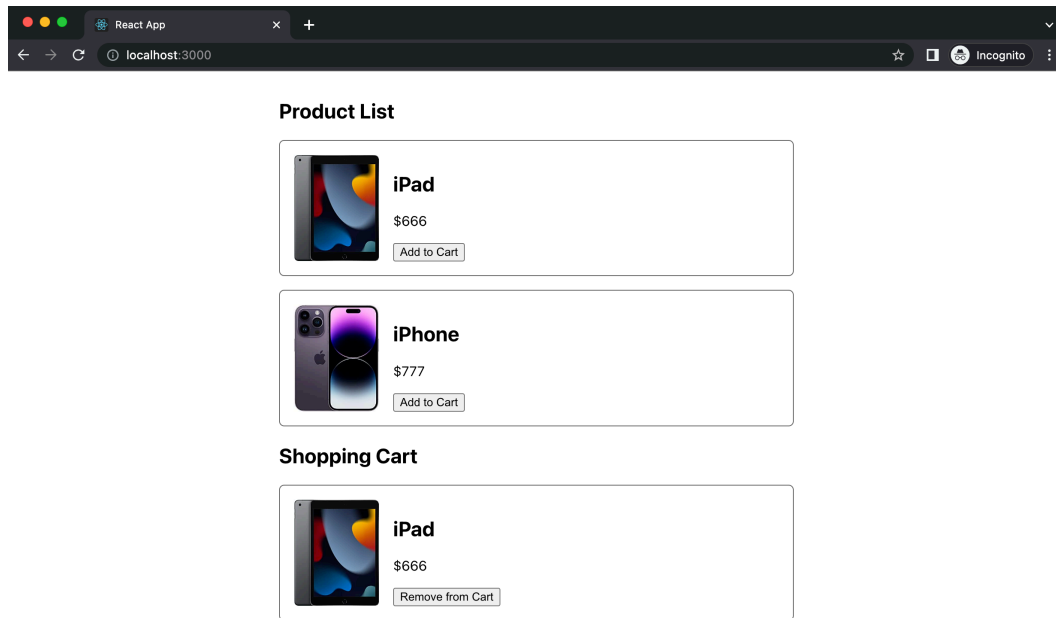


Figure 4.1: The product list page

The `ProductList` component will display the product's image, name, and price, with an **Add to Cart** button, while the `Cart` component will display a list of cart items with a **Remove from Cart** button.

A naive implementation of `ProductList` might look like this:

```
type Product = {
  id: string;
  name: string;
  image: string;
  price: number;
};

const ProductList = ({
  products,
```

```

    addToCart,
  }: {
    products: Product[];
    addToCart: (id: string) => void;
  }) => (
    <div>
      <h2>Product List</h2>
      {products.map((product) => (
        <div key={product.id} className="product">
          <img src={product.image} alt={product.name} />
          <div>
            <h2>{product.name}</h2>
            <p>${product.price}</p>
            <button onClick={() => addToCart(product.id)}>Add to Cart
              </button>
          </div>
        </div>
      ))}
    </div>
  );

export default ProductList;

```

This functional component named `ProductList` accepts two props: `products` (an array of product objects) and `addToCart` (a function used to add a product to a shopping cart).

Each product object is of the `Product` type, which has the `id`, `name`, `image`, and `price` properties.

The component maps over the `products` array and renders a `div` for each product, including an image, product name, price, and an **Add to Cart** button. When the **Add to Cart** button is clicked, the `addToCart` function is invoked with the corresponding product's `id` prop as an argument.

The `Cart` component has a similar structure, as you can imagine; it requires a list of items and renders a button with the text **Remove from Cart**, and a callback function for the user to invoke:

```

const Cart = ({
  cartItems,
  removeFromCart,
}): {
  cartItems: Product[];
  removeFromCart: (id: string) => void;
}) => (
  <div>
    <h2>Shopping Cart</h2>
    {cartItems.map((item) => (

```

```

    <div key={item.id} className="product">
      <img src={item.image} alt={item.name} />
      <div>
        <h2>{item.name}</h2>
        <p>${item.price}</p>
        <button onClick={() => removeFromCart(item.id)}>
          Remove from Cart
        </button>
      </div>
    </div>
  )}
</div>
);

```

The `Cart` component iterates over the `cartItems` array and, for each item, it renders a `div` with the item's image, name, price, and a **Remove from Cart** button. When this button is clicked, it invokes the `removeFromCart` function with the respective item's `id` prop as an argument, signifying that this item should be removed from the cart.

To reduce the duplication and make each component only do one thing, we can extract a `LineItem` component:

```

import { Product } from "../types";

const LineItem = ({
  product,
  performAction,
  label,
}): {
  product: Product;
  performAction: (id: string) => void;
  label: string;
}) => {
  const { id, image, name, price } = product;

  return (
    <div key={id} className="product">
      <img src={image} alt={name} />
      <div>
        <h2>{name}</h2>
        <p>${price}</p>
        <button onClick={() => performAction(id)}>{label}</button>
      </div>
    </div>
  )
}

```

```
    );  
  };  
  
  export default LineItem;
```

We defined a functional component called `LineItem` that renders a product's details and a button. It accepts properties for `product`, `performAction`, and `label`, and uses destructuring to extract the necessary values. The component returns JSX code to display the product's information and trigger the `performAction` function when the button is clicked.

For the `ProductList` and `Cart` components, you can simply pass in different props to the `LineItem` component to reduce the duplication we had before:

```
const ProductList = ({  
  products,  
  addToCart,  
}: {  
  products: Product[];  
  addToCart: (id: string) => void;  
}) => (  
  <div>  
    <h2>Product List</h2>  
    {products.map((product) => (  
      <LineItem  
        key={product.id}  
        product={product}  
        performAction={addToCart}  
        label="Add to Cart"  
      />  
    ))}  
  </div>  
);
```

The new `ProductList` component receives `products` and `addToCart` as props. It renders a list of products, with each product having an **Add to Cart** button.

Similarly, for the `Cart` component, we'll have a similar structure that reuses the `LineItem` component for rendering the product details (image, name, and price):

```
const Cart = ({  
  cartItems,  
  removeFromCart,  
}: {  
  cartItems: Product[];  
  removeFromCart: (id: string) => void;  
}) => (  
  <div>  
    <h2>Cart</h2>  
    {cartItems.map((product) => (  
      <LineItem  
        key={product.id}  
        product={product}  
        performAction={removeFromCart}  
        label="Remove from Cart"  
      />  
    ))}  
  </div>  
);
```

```
} => (  
  <div>  
    <h2>Shopping Cart</h2>  
    {cartItems.map((item) => (  
      <LineItem  
        key={item.id}  
        product={item}  
        performAction={removeFromCart}  
        label="Remove from Cart"  
      />  
    ))}  
  </div>  
);
```

This is a more maintainable and reusable approach by following the DRY principle. There is less chance of introducing bugs because changes only need to be made in one place, and if we must add new features of `LineItem`, we only need to touch a single component.

In this section, we dove into the DRY principle. It guided us to eliminate redundancy in our code, reducing the likelihood of inconsistencies and bugs. By avoiding code duplication, we've simplified maintenance as changes in functionality need to be addressed in a single place. As we refined our understanding of DRY, we prepared ourselves to enhance our component structure using a key concept in React – composition.

Using composition

In React, **composition** is a natural pattern of the component model. For instance, JSX's markup language syntax enables us to pair a `div` with an `h2` tag seamlessly, without the need to incorporate anything new.

The custom component is nothing more special than the built-in ones like a `div`; you can use your `Cart` component with a `div` just as you can use a `p` tag with it. This pattern enables more straightforward reuse of components, which can contribute to cleaner and more maintainable code.

Let's consider an example. Suppose we're building a `UserDashboard` component that displays user information. The profile includes an avatar, a name, and a list of the user's friends and the latest posts. Here's how it might look:

```
type User = {  
  name: string;  
  avatar: string;  
  friends: string[];  
};  
  
type Post = {
```

```
    author: string;
    summary: string;
  };

type UserDashboardProps = {
  user: User;
  posts: Post[];
};

function UserDashboard({ user, posts }: UserDashboardProps) {
  return (
    <div>
      <h1>{user.name}</h1>
      <img src={user.avatar} alt="profile" />
      <h2>Friends</h2>
      <ul>
        {user.friends.map((friend) => (
          <li key={friend}>{friend}</li>
        ))}
      </ul>
      <h2>Latest Posts</h2>
      {posts.map((post) => (
        <div key={post.author}>
          <h3>{post.author}</h3>
          <p>{post.summary}</p>
        </div>
      ))}
    </div>
  );
}

export default UserDashboard;
```

In this simplified example, `UserDashboard` is responsible for rendering the user's profile, a list of friends, and the latest posts, which violates the SRP. We can break it down into smaller components, each responsible for one thing.

First, we can extract the profile-related JSX into a `UserProfile` component that displays the user's profile, which includes an `h1` tag (the user's name) and an avatar image:

```
const UserProfile = ({ user }: { user: User }) => {
  return (
    <>
      <h1>{user.name}</h1>
```

```

        <img src={user.avatar} alt="profile" />
      </>
    );
  };
};

```

Next, we create a `FriendList` component that displays a list of the user's friends; it includes an `h2` tag and a friends list:

```

const FriendList = ({ friends }: { friends: string[] }) => {
  return (
    <>
      <h2>Friends</h2>
      <ul>
        {friends.map((friend) => (
          <li key={friend}>{friend}</li>
        ))}
      </ul>
    </>
  );
};

```

Finally, we create a `PostList` component that displays a feed of posts, which includes an `h2` tag and a list of posts:

```

const PostList = ({ posts }: { posts: Post[] }) => {
  return (
    <>
      <h2>Latest Posts</h2>
      {posts.map((post) => (
        <div key={post.author}>
          <h3>{post.author}</h3>
          <p>{post.summary}</p>
        </div>
      ))}
    </>
  );
};

```

Now, our `UserDashboard` component becomes simpler and delegates its responsibilities to these smaller components:

```

function UserDashboard({ user, posts }: UserDashboardProps) {
  return (
    <div>
      <UserProfile user={user} />

```

```
    <FriendList friends={user.friends} />
    <PostList posts={posts} />
  </div>
);
}
```

The refactored version of the `UserDashboard` component is superior due to several reasons:

- **Separation of Concerns:** By separating different parts of the component (user profile, friend list, and post list) into their own components (`UserProfile`, `FriendList`, and `PostList`), you ensure that each component is responsible for one task. This improves the maintainability of the code.
- **Readability:** The new version of `UserDashboard` is easier to read and understand. It becomes immediately clear what this component is rendering: a user profile, a list of friends, and a list of posts. There's no need to read through details about how each of these parts is rendered.
- **Reusability:** The `UserProfile`, `FriendList`, and `PostList` components can now be reused in other parts of the application if needed, thereby promoting code reuse and reducing redundancy.
- **Testability:** Smaller, single-responsibility components are easier to test, as they tend to have less complex interactions and dependencies. We'll cover the testing and testability in *Chapter 5*.

This is a simple example, but it illustrates the basic idea of composition in React. Composition can become more complex as you deal with components that have their own state or logic, but the core principle remains the same: building larger components from smaller, reusable parts.

This section brought us to the power of composition in React. With composition, we could efficiently structure and combine our components, creating complex user interfaces from simpler, single-responsibility components. We observed how composition allowed us to fully leverage the principles of SRP and DRY, leading to the creation of sophisticated UIs that remain easy to understand, test, and maintain.

Combining component design principles

We've separately analyzed the principles of single responsibility, don't repeat yourself, and composition. However, in practical coding scenarios, things can get intricate, necessitating the simultaneous application of multiple principles to enhance the readability and maintainability of the code.

Let's consider an example of a `Page` component, which might have many responsibilities such as managing the state and behavior of a header, sidebar, and main content area. Also, there are a bunch of props for configuring each of these sections. It's typical to encounter such code when individuals simply replicate the existing code base without much consideration or critical thinking; so the prop list grows as new features are added.

Here is a simplified example:

```
import React from "react";

type PageProps = {
  headerTitle: string;
  headerSubtitle: string;
  sidebarLinks: string[];
  isLoading: boolean;
  mainContent: React.ReactNode;
  onHeaderClick: () => void;
  onSidebarLinkClick: (link: string) => void;
};

function Page({
  headerTitle,
  headerSubtitle,
  sidebarLinks,
  mainContent,
  isLoading,
  onHeaderClick,
  onSidebarLinkClick,
}: PageProps) {
  return (
    <div>
      <header onClick={onHeaderClick}>
        <h1>{headerTitle}</h1>
        <h2>{headerSubtitle}</h2>
      </header>
      <aside>
        <ul>
          {sidebarLinks.map((link) => (
            <li key={link} onClick={() => onSidebarLinkClick(link)}>
              {link}
            </li>
          ))}
        </ul>
      </aside>
      {!isLoading && <main>{mainContent}</main>}
    </div>
  );
}
```

We defined a `Page` component; the component uses the preceding props to render a page with a clickable header, a sidebar containing clickable links, and a main content section. The component expects an object of the `PageProps` type as its properties.

Let's take a closer look inside `PageProps`:

- `headerTitle`: This string will be displayed as the main title in the page's header
- `headerSubtitle`: This string will be displayed as the subtitle in the page's header
- `sidebarLinks`: This is an array of strings, where each string represents a link that will be displayed in the page's sidebar
- `isLoading`: This is a flag to determine whether the main content is ready or not
- `mainContent`: This can be any valid React node (a component, an element, null, etc.) that represents the main content of the page
- `onHeaderClick`: This function will be executed when the header section of the page is clicked
- `onSidebarLinkClick`: This function will be executed when any sidebar link is clicked. The function will receive the clicked link as an argument

The `Page` component has multiple responsibilities, and it has a long list of props that could make it hard to work with. A long prop list – when a component has over five props – typically signals a need for component breakdown. This is because remembering the purpose of each prop can be challenging, and it also increases the likelihood of passing the wrong prop or misordering them.

We can group the props based on how they are used. The `headerTitle`, `headerSubtitle`, and `onHeaderClick` props can be split out into a group, while `isLoading` and `mainContent` belong to another group.

Extracting a small part of the large component is always a good starting point. Note that there might be many ways to do the extracting; if the information looks related to each other, we can group them and create a new component for that group of data. For example, we can extract a `Header` component first:

```
type HeaderProps = {
  headerTitle: string;
  headerSubtitle: string;
  onHeaderClick: () => void;
};

const Header = ({
  headerTitle,
  headerSubtitle,
  onHeaderClick,
}: HeaderProps) => {
  return (
```

```
    <header onClick={onHeaderClick}>
      <h1>{headerTitle}</h1>
      <h2>{headerSubtitle}</h2>
    </header>
  );
};
```

This Header component in React takes three props – `headerTitle`, `headerSubtitle`, and `onHeaderClick` – and renders a header with the provided title and subtitle. The `onHeaderClick` prop is then called when the header is clicked.

Now, because `title`, `subtitle`, and the `onClick` callback are already in the Header component, we don't need the `header` prefix in the prop names. Let's rename these props:

```
type HeaderProps = {
  title: string;
  subtitle: string;
  onClick: () => void;
};

const Header = ({
  title,
  subtitle,
  onClick,
}: HeaderProps) => {
  return (
    <header onClick={onClick}>
      <h1>{title}</h1>
      <h2>{subtitle}</h2>
    </header>
  );
};
```

It's now much clearer regarding what Header does – it accepts `title`, `subtitle`, and `onClick` and doesn't need to know anything beyond that. This extraction also increases the reusability of Header, meaning we might reuse the component in a different place.

Now, we can extract a Sidebar component with the same approach:

```
type SidebarProps = {
  links: string[];
  onLinkClick: (link: string) => void;
};

const Sidebar = ({ links, onLinkClick }: SidebarProps) => {
```

```
    return (
      <aside>
        <ul>
          {links.map((link) => (
            <li key={link} onClick={() => onLinkClick(link)}>
              {link}
            </li>
          ))}
        </ul>
      </aside>
    );
  };
};
```

The Sidebar component accepts an array of `links` and an `onLinkClick` function as props and generates a list of clickable items from the `links` array. The `onLinkClick` function is triggered when a link is clicked, passing the clicked link as an argument.

After we extracted Header and Sidebar, the only thing left in `Page` is the part related to the main content. We can apply the same approach for the main content, extracting a `Main` component with the simple JSX fragment, as in the following code:

```
type MainProps = {
  isLoading: boolean;
  content: React.ReactNode;
};

const Main = ({ isLoading, content }: MainProps) => {
  return <>{!isLoading && <main>{content}</main>}</>;
};
```

As we extracted most of the content out of the `Page` component, we can now use these simple components without changing the public interface of `Page`:

```
function Page({
  headerTitle,
  headerSubtitle,
  sidebarLinks,
  mainContent,
  isLoading,
  onHeaderClick,
  onSidebarLinkClick,
}: PageProps) {
  return (
    <div>
      <Header
```

```

        title={headerTitle}
        subtitle={headerSubtitle}
        onClick={onHeaderClick}
      />
      <Sidebar links={sidebarLinks} onLinkClick={onSidebarLinkClick}
      />
      <Main isLoading={isLoading} content={mainContent} />
    </div>
  );
}

```

The `Page` component arranges the `Header`, `Sidebar`, and `Main` components and takes in several props. It then passes these props to the respective child components – `Header` gets the title, subtitle, and a click handler; `Sidebar` receives a list of links and a click handler; and the `Main` component gets the main content and a loading state.

The refactored `Page` looks much nicer, but it's not perfect. Let's consider a common issue with the current code. What happens if we need to pass in new props to `Sidebar` or `Main`?

To accept the new props passed into `Sidebar` or `Main`, we need to extend the prop list, which already has seven props. For people who use the `Page` component, as more props are added, they will have to remember even more props, which wouldn't be a good experience (not to mention the additional testing efforts because of these props).

Instead of accepting these detailed descriptions to customize `Header` or `Sidebar`, we can pass in an instance of `Header` and then just plug it into the correct slot (to replace the `Header` component):

```

type PageProps = {
  header: React.ReactNode;
  sidebarLinks: string[];
  isLoading: boolean;
  mainContent: React.ReactNode;
  onSidebarLinkClick: (link: string) => void;
};

function Page({
  header,
  sidebarLinks,
  mainContent,
  isLoading,
  onSidebarLinkClick,
}: PageProps) {
  return (
    <div>
      {header}

```

```
    <Sidebar links={sidebarLinks}
      onLinkClick={onSidebarLinkClick} />
    <Main isLoading={isLoading} content={mainContent} />
  </div>
);
}
```

Now, the `Page` component accepts a `header` prop (along with a list of sidebar links, a loading state, main content, and a link click handler as props) and renders the `Header` component directly. That means we can pass in any `Header` instance from outside of `Page`.

Similarly, we can do the same for `Sidebar` and `Main`:

```
type PageProps = {
  header: React.ReactNode;
  sidebar: React.ReactNode;
  main: React.ReactNode;
};

function Page({ header, sidebar, main }: PageProps) {
  return (
    <div>
      {header}
      {sidebar}
      {main}
    </div>
  );
}
```

The `Page` component accepts three props – `header`, `sidebar`, and `main` – with each expected to be a **ReactNode**, a type that includes practically anything that can be rendered in React (strings, elements, arrays, etc.). The `Page` component simply renders these three props in a `div` in the order they are provided, effectively forming a simple page layout with a header, a sidebar, and a main section.

You can then use `Page` in the most flexible way – you can pass in a fully customized `Header`, `Sidebar`, and `Main` as parameters into the `Page` component:

```
const MyPage = () => {
  return (
    <Page
      header={
        <Header
          title="My application"
          subtitle="Product page"
          onClick={() => console.log("toggle header")}
        />
      />
  );
}
```

```
    />
  }
  sidebar={
    <Sidebar
      links={["Home", "About", "Contact"]}
      onLinkClick={() => console.log("toggle sidebar")}
    />
  }
  main={<Main isLoading={false}
    content={<div>The main</div>} />}
/>
);
};
```

This `MyPage` component renders a `Page` component, passing in `Header`, `Sidebar`, and `Main` components as props, while click events on the `Header` and `Sidebar` components will log certain messages to the console.

Note that here you can pass anything into `Page` to define header, sidebar, or main. Here's an example:

```
const MySimplePage = () => {
  return (
    <Page
      header={
        <h1>A simple header</h1>
      }
      sidebar={
        <aside>
          <ul>
            <li>Home</li>
            <li>About</li>
          </ul>
        </aside>
      }
      main={<div>The main content</div>}
    />
  );
};
```

The `MyPage` component wraps a `Page` component. The `Page` component receives three props: `header`, `sidebar`, and `main`, each containing JSX elements that specify what's to be rendered in each respective section of the page. The `header` prop has a heading, the `sidebar` prop includes a list with **Home** and **About** items, and the `main` section contains the main content of the page.

The original `Page` component was heavily burdened with a multitude of responsibilities, leading to a long list of props. This design presented a **prop drilling** problem, where a large amount of data had to be passed down through multiple layers of components. This setup was both complex and hard to maintain.

With that, let's review our process of how we applied different principles. The refactoring process started by breaking down the monolithic `Page` component into smaller, more manageable components – `Header`, `Sidebar`, and `Main` – using the SRP. These sub-components were designed to handle their respective responsibilities, thus simplifying their individual prop requirements.

Once these components were extracted, we modified the `Page` component to accept these sub-components (`Header`, `Sidebar`, and `Main`) as props using composition. This approach significantly reduced the prop drilling issue, as each sub-component now receives props directly at the point of usage.

This refactoring exercise streamlined the `Page` component, resulting in a cleaner, more manageable code base. It utilized the principles of component composition and single responsibility to solve the prop drilling problem effectively.

Summary

This chapter covered several key principles in designing and developing components in React: the SRP, DRY, and the use of component composition. Each of these principles provides different strategies for achieving clean, maintainable, and scalable code bases.

By understanding and applying these principles, we can create a solid foundation for our React applications. These strategies lead to more organized, scalable, and robust code bases, ultimately making our work as developers more effective and enjoyable.

In the next chapter, we'll start to look into an exciting topic in React applications – testing – and see how good structured tests can protect us from making mistakes while helping us to improve the code quality.

Part 2:

Embracing

Testing Techniques

In this part, you will delve into the significance of testing in frontend development, exploring various testing methodologies and refactoring techniques that will ensure the robustness and maintainability of your React applications.

This part contains the following chapters:

- *Chapter 5, Testing in React*
- *Chapter 6, Exploring Common Refactoring Techniques*
- *Chapter 7, Introducing Test-Driven Development with React*

5

Testing in React

Welcome to this immersive chapter on testing in React. In this chapter, we will learn the importance of software testing, understand the different types of testing – including unit, integration, and **end-to-end (E2E)** testing – and delve into the use of popular testing tools such as Cypress, Jest, and the React Testing Library. In addition, we will demystify concepts such as stubbing and mocking, ensuring you are well-equipped to handle complex testing scenarios.

Our overarching goal is to foster a solid understanding of testing strategies and their implementation in React. We aim to enhance your ability to write tests that make your application resilient to bugs and regressions and ensure the seamless addition of new features.

By the end of this chapter, you'll have a comprehensive understanding of React testing and be ready to implement efficient testing practices in your projects. So, let's get started and step into the exciting world of React testing!

In this chapter, we will cover the following topics:

- Understanding why we need tests
- Learning about different types of tests
- Testing individual units with Jest
- Learning about integration tests
- Learning about E2E tests using Cypress

Technical requirements

A GitHub repository has been created to host all the code we'll discuss in this book. For this chapter, you can find the recommended structure at <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch5>.

Understanding why we need tests

Testing is not just an optional best practice; it's a critical part of building reliable and maintainable software. Without tests, you're essentially navigating the complex seas of software development without a compass. Let's understand the multiple benefits that testing brings:

- **Ensuring code correctness:** Tests serve as a seal of validation that your code performs exactly the way it's supposed to. A well-written test verifies that your functions return the expected output for a given input, your components render correctly, and your application behaves as anticipated.
- **Preventing regression:** As applications grow and evolve, new code can sometimes unintentionally break existing functionality. This is known as a **regression**. Automated tests act as a safety net, catching these regressions before they reach production.
- **Facilitating refactoring and maintenance:** Fear often surrounds the process of refactoring or updating legacy code. Tests alleviate this fear. They provide a comfort zone, assuring that if you accidentally break something during the update or refactoring process, your tests will catch it.
- **Boosting confidence in code quality:** Tests elevate the confidence level of your team. When a suite of well-written tests backs your code, you have a quantifiable measure of your code's quality. This assurance is especially beneficial when you're adding new features or making changes to the system.
- **Documentation:** Tests also serve as a form of documentation. They provide a clear understanding of what a function or component is supposed to do, helping new developers on the team understand the project's functionality.

In the subsequent sections, we'll delve deeper into the various types of testing you'll commonly use in React applications and learn how to use testing tools effectively. Buckle up for an engaging ride into the realm of software testing.

Learning about different types of tests

Testing, in the realm of software development, isn't a one-size-fits-all approach. Rather, it is categorized into different types, each serving a distinct purpose and offering unique insights into the functionality and reliability of the application. It is important to understand these categories to ensure the overall health and robustness of your application. Typically, you will have unit tests, integration tests, and E2E tests in one code base.

We will define each type here briefly and discuss each type in detail in the following sections:

- **Unit tests:** These tests focus on testing individual components or functions in isolation to ensure they work as expected

- **Integration tests:** These tests examine the interactions between different modules or services to verify they work together cohesively
- **E2E tests:** These tests test the entire application flow from start to finish, mimicking real-world user behavior to validate that the system works as a whole

How you structure your tests in a project also matters. For example, you should have a lot of unit tests that run fast and can provide detailed feedback, and you should have only a few E2E tests to ensure all the parts work together. This approach aligns with the principles of the test pyramid.

Originally conceived by Mike Cohn, the **test pyramid** recommends having a larger number of unit tests compared to integration or E2E tests. The reasoning is simple – unit tests are quicker, simpler, and more cost-effective to maintain:

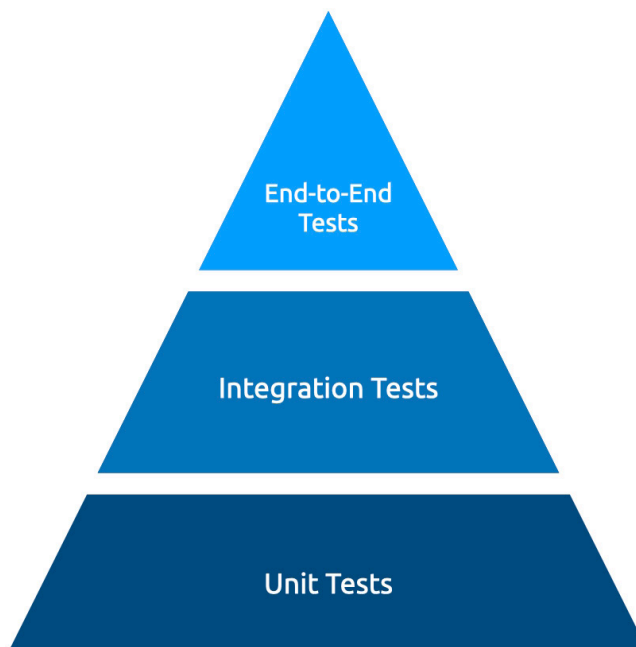


Figure 5.1: The traditional test pyramid

However, in the modern frontend world, this model is evolving. More value is being placed on integration and E2E tests due to the increasing complexity and interactivity of frontend applications. Tools such as Cypress and Puppeteer facilitate writing E2E tests that emulate user behavior on the browser, while libraries such as the React Testing Library encourage more integration tests by making it easier to test component interactions.

New types of tests are also introduced in frontend applications. The visual regression tests is one of them. **Visual regression testing** is a method of testing in which the visual aspects of a web application are captured and compared to previous states or versions. This type of testing is particularly useful in catching unintended visual bugs and changes in a user interface that may be introduced during development.

Visual regression tests work by taking screenshots (or snapshots) of web pages or components at different stages, and then comparing these screenshots pixel by pixel to identify any visual differences. When a difference is detected, it is flagged for review. The review can then determine if the change is expected (due to a new feature or design update) or if it's an unintended regression that needs to be fixed.

In frontend testing, **static checks** involve analyzing code without executing it to identify errors and ensure coding standards. This includes checking for syntax errors, enforcing coding style through linting, verifying correct data types with type checking, analyzing code complexity, examining dependencies, and identifying security vulnerabilities.

The exact shape of your test pyramid might vary depending on your application's needs, but the critical takeaway is to have a balanced testing strategy that provides quick and useful feedback at different levels of your application:

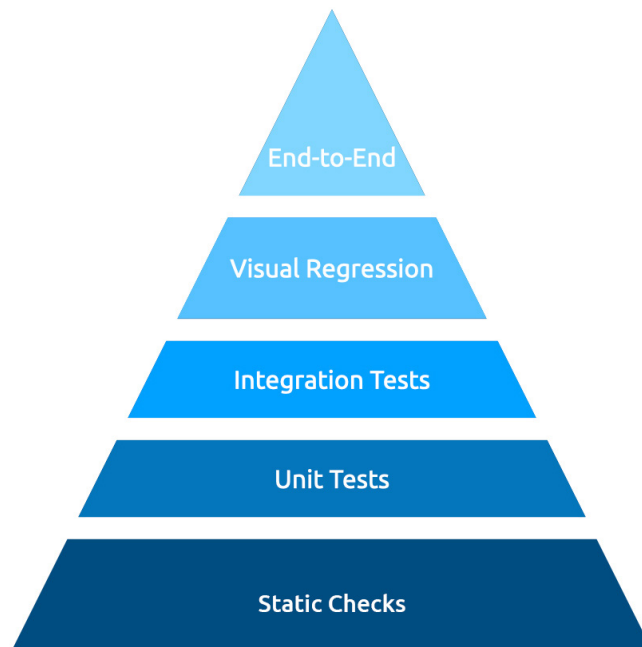


Figure 5.2: The enhanced test pyramid

The next sections in this chapter will give you hands-on experience in writing these types of tests for a React application, ensuring that you are well-equipped to bring these concepts into your projects. Let's forge ahead!

Testing individual units with Jest

Unit tests are the smallest and most foundational part of the testing pyramid, verifying the behavior of individual units of code in isolation, such as functions, methods, or components. These tests are quick to write and execute, offering immediate feedback to developers.

We'll use Jest to write unit tests and integration tests in this book. Jest is a comprehensive JavaScript testing framework built by Facebook, with a strong focus on simplicity. It's feature-rich and supports asynchronous testing, mocking, and snapshot testing, making it a great choice for React applications.

Writing your first test

Let's write a simple test. Say you have an `add` function in a file called `math.ts`:

```
export function add(a: number, b: number) {  
  return a + b;  
}
```

To test this function, you must create a `math.test.ts` file in the same directory:

```
import { add } from './math';  
  
test('add adds numbers correctly', () => {  
  expect(add(1, 2)).toBe(3);  
});
```

You've now written your first test! The `test` function takes two arguments: a string description of the test and a callback function that implements the test. `expect` is a Jest function that takes the actual value, and `toBe` is a matcher function that compares the actual value with the expected value.

Another way of writing a test is to use the `it` function. In Jest, `test` and `it` are actually the same function and can be used interchangeably; the names just come from different testing conventions:

- `test`: This is a common name for a test function in many testing frameworks and languages. If you come from a background of using other testing libraries, you might find `test` to be more intuitive or familiar.
- `it`: This comes from **behavior-driven development (BDD)** style frameworks such as Jasmine or Mocha.

The idea of using `it` is to make the tests read more like sentences. For example, `it('adds 1 + 2 to equal 3', () => expect(1 + 2).toBe(3))` reads like "it adds 1 + 2 to equal 3."

Note

BDD is a software development approach that emphasizes collaboration between developers, QA, and non-technical participants in a software project. It highlights the need to start with a clear understanding of desired behavior before development starts, thereby aligning development with business needs.

BDD encourages expressing software behaviors in plain, descriptive language that can be read and understood by all stakeholders. It leverages executable specifications, often written in a language such as Gherkin, that guide development and serve as acceptance criteria.

BDD aims to reduce misunderstandings by encouraging collaboration, making the behavior of a system explicit and understandable by all, and ensuring that the software developed truly meets the needs of the business.

It's a matter of team preference and what aligns best with your team's testing philosophy – some teams prefer the sentence-like structure that it provides as it often makes it clearer what a test is trying to verify, especially to non-developers, while others might find `test` to be more straightforward and less verbose. We're going to write tests that follow the BDD style in this book.

Grouping tests

Grouping related tests in one block significantly improves the readability of your test files. By clearly delineating different areas of functionality, a block allows anyone reading the tests to understand the context of the test suite at a glance. This enhanced understanding is crucial for comprehension of what functionality is being verified. In a large code base, with numerous tests, this organization can greatly decrease the cognitive load required to understand how different parts of the application are tested.

In Jest, we can use the `describe` function to group related tests into one unit. For instance, consider a function, `add`, that includes multiple cases: the addition of negative numbers, the combination of one negative and one positive number, decimal sums, or even computations involving imaginary numbers. It would be prudent to gather all these distinct cases under one `describe` block, like so:

```
import { add } from './math';

describe('math functions', () => {
  it('adds positive numbers correctly', () => {
    expect(add(1, 2)).toBe(3);
  });

  it('adds negative numbers correctly', () => {
    expect(add(-1, -2)).toBe(-3);
  });

  // More tests...
});
```

The `describe` function is used to group related tests – in this case, tests for some math functions. Within this group, there are two `it` functions, each representing a single test. The first test checks if the `add` function correctly adds two positive numbers, and the second test checks if the `add` function correctly adds two negative numbers.

With Jest, you can nest `describe` blocks to organize your tests more systematically. For example, suppose we're expanding our suite to include subtraction, multiplication, and division in our calculator functionality. We can structure our test suite in the following manner:

```
describe('calculator', () => {
  describe('addition', () => {
    it('adds positive numbers correctly', () => {
      expect(add(1, 2)).toBe(3);
    });

    it('adds negative numbers correctly', () => {
      expect(add(-1, -2)).toBe(-3);
    });

    // More tests...
  })

  describe('subtraction', () => {
    it('subtracts positive numbers', () => {});
  })

  // Other describe blocks for multiplication and division
});
```

In this code snippet, we have a top-level `describe` block labeled `calculator`. Within this block, we have nested `describe` blocks for each mathematical operation. For instance, in the `addition` block, we have individual `it` tests for different scenarios of adding numbers. Similarly, we start a new `describe` block for `subtraction`. This nested structure makes our test suite more organized, readable, and easier to navigate, particularly when dealing with a large number of tests or complex scenarios.

Testing React components

As we mentioned previously, Jest is a great tool for testing different types of applications, and it supports React applications out of the box. Although it's possible to use Jest alone, it would be a bit more cumbersome and verbose than using a dedicated library such as the **React Testing Library**.

The React Testing Library is a lightweight yet powerful library for testing React components. It's built on top of the popular JavaScript testing framework, Jest, and adds specific utilities for working with React components. The philosophy of the React Testing Library is to encourage writing tests that closely

resemble how your software is used. It encourages you to interact with your app just like how users would, meaning you test the functionality and not the implementation details. This approach leads to more robust and maintainable tests that will give you confidence that your app will work in production.

In the code provided in this book, the project has already been set up for you with the React Testing Library. Simply clone the code mentioned in the *Technical requirements* section into your local directory and you are good to go.

All right – let's start with a simple React component to see how we can test it with the React Testing Library. The `Section` component is a presentational component that accepts two props, `heading` and `content`, and renders the props in an `article` tag:

```
type SectionProps = {
  heading: string;
  content: string;
};

const Section = ({ heading, content }: SectionProps) => {
  return (
    <article>
      <h1>{heading}</h1>
      <p>{content}</p>
    </article>
  );
};

export { Section };
```

To test the component, we can create a new file next to `Section.tsx`, which we will call `Section.test.tsx`. This is where our test code will live. Then, we'll use the React Testing Library to check the `Section` component:

```
import React from "react";
import { render, screen } from "@testing-library/react";

import { Section } from "../component/Section";

describe("Section", () => {
  it("renders a section with heading and content", () => {
    render(<Section heading="Basic" content="Hello world" />);

    expect(screen.getByText("Basic")).toBeInTheDocument();
    expect(screen.getByText("Hello world")).toBeInTheDocument();
  });
});
```

This test code makes use of `@testing-library/react` to verify that the `Section` component behaves as expected – the text **Basic** and **Hello world** should be present in the HTML document. The render function provided by `@testing-library/react` is used to render the `Section` component with specific props: a heading of **Basic** and content of **Hello world**.

Following the rendering, the `screen.getByText` function is used to query the DOM (which represents the rendered output of the `Section` component) for elements containing specific text.

Next, `expect` and `toBeInTheDocument` are then used to make assertions about the state of these elements. Specifically, the test is asserting that there is an element with the **Basic** text and an element with the **Hello world** text present in the DOM, which would indicate that the `Section` component has correctly rendered its heading and content props.

This straightforward unit test for a React component serves as a useful starting point. However, in complex real-world projects, we often encounter scenarios where multiple components need to interact harmoniously. For instance, consider a checkout page that integrates an address collection component, a payment component, and a price calculation logic component.

To confidently ensure the seamless interaction of these distinct components, we must employ a more comprehensive testing strategy: integration tests.

Learning about integration tests

Integration tests are positioned above unit tests in the pyramid, validating the interactions between multiple units of code. These could be component interactions or interactions between the client side and server side. Integration tests aim to identify issues that may arise when different parts of the system are combined.

One such scenario involves testing the interaction between two separate components to verify that they function correctly together – this is integration testing at a UI component level. Additionally, if you're looking to ensure smooth collaboration between your frontend code and backend services, the tests you write for this purpose would also be classified as integration tests, which verify that different layers of your application are working correctly together.

Let's have a look at an example of an integration test for a React component. In *Figure 5.3*, there is a **Terms and Conditions** section, which includes a long text about the legal information, and a checkbox for the user to consent. There is also a **Next** button, which is disabled by default. However, once the user selects **Accept the Terms and Conditions**, the button will be enabled, and the user can proceed:

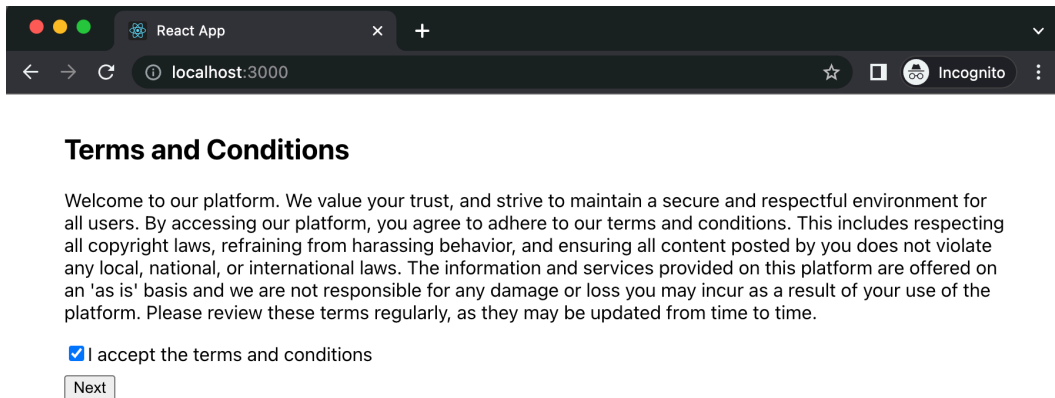


Figure 5.3: The Terms and Conditions component

This integration test can be described using the following code snippet – we are not testing the **Checkbox** and **Next** buttons separately, but verifying the interaction between them:

```
describe('Terms and Conditions', () => {
  it('renders learn react link', () => {
    render(<TermsAndConditions />);
    const button = screen.getByText('Next');
    expect(button).toBeDisabled();

    const checkbox = screen.getByRole('checkbox');

    act(() => {
      userEvent.click(checkbox);
    })

    expect(button).toBeEnabled();
  });
})
```

The `describe` function is used to group all tests associated with the `TermsAndConditions` component, forming a so-called test suite. Within this suite, we have a single test case denoted by the `it` function. The description of this test is **renders learn react link**, which appears to be a misnomer considering the operations carried out in this test. A more suitable description might be **Enables the next button upon accepting terms and conditions**.

Initially, the `render` function is invoked to display the `TermsAndConditions` component. This function produces a series of output, or render results, that can be queried in various ways to assess whether the component behaves as expected.

We then try to find a button by its text, **Next**, using the `screen.getByText` function – which returns the element found on the page. At this point, we expect this button to be disabled, so we confirm this expectation by calling `expect(button).toBeDisabled()`.

Next, we look for the checkbox using the `screen.getByRole` function. This function allows us to find the checkbox based on its role, which is `checkbox`.

The user interaction of ticking the checkbox is simulated using the `userEvent.click` function, which is wrapped in React's `act` function. The `act` function ensures that all updates related to these actions are processed and applied before moving forward; this way, our assertions will examine the component in its updated state.

Finally, we verify that the button is enabled after the checkbox has been clicked. This is done using `expect(button).toBeEnabled()`. If this statement holds true, we know that our component behaves as intended – that is, disabling the **Next** button until the user accepts the terms and conditions.

Now, let's look at how the code is written. The `TermsAndConditions` component under test is composed of a few components – `heading`, `LegalContent`, and `UserConsent`. Plus, `UserConsent` itself is composed of `CheckBox` and `Button`:

```
const TheLegalContent = () => {
  return (
    <p>
      { /* ... */ }
    </p>
  );
};

type CheckBoxProps = {
  label: string;
  isChecked: boolean;
  onCheck: (event: any) => void
}

const CheckBox = ({label, isChecked, onCheck}: CheckBoxProps) => {
  return (
    <label>
      <input
        type="checkbox"
        checked={isChecked}
        onChange={onCheck}
      />
      {label}
    </label>
  )
}
```

```
}

type ButtonProps = {
  type: 'standard' | 'primary' | 'secondary';
  label: string;
  disabled?: boolean;
}

const Button = ({label, disabled = true}: ButtonProps) => {
  return (
    <div style={{margin: '0.5rem 0'}}>
      <button disabled={disabled}>{label}</button>
    </div>
  )
}

const UserConsent = () => {
  const [isChecked, setIsChecked] = useState(false);

  const handleCheckboxChange = (event: React.
    ChangeEvent<HTMLInputElement>) => {
    setIsChecked(event.target.checked);
  };

  return (
    <>
      <CheckBox isChecked={isChecked} onCheck={handleCheckboxChange}
        label="I accept the terms and conditions" />
      <Button type="primary" label="Next" disabled={!isChecked} />
    </>
  );
};

const TermsAndConditions = () => {
  return (
    <div>
      <h2>Terms and Conditions</h2>
      <TheLegalContent />
      <UserConsent />
    </div>
  );
};

export { TermsAndConditions };
```

The sole component that gets exported in this code is `TermsAndConditions`, which is the primary subject of our testing strategy. In our tests, we employ `userEvent.click` to initiate a click event within the realm of a `jsdom` environment.

Essentially, our focus is not on testing the isolated React components (such as `CheckBox` and `Button`; they should have their own unit tests), but rather on the DOM elements and their interactions. It's important to clarify that we're not invoking a full-fledged browser here, but rather a headless `jsdom` variant that exists in memory. Yet, despite the simulated environment of these integration tests, they still provide us with the confidence that the click event and button enablement are functioning as expected.

Note

jsdom is a JavaScript-based headless browser that can be used to create a realistic testing environment that simulates a web browser's environment. It is an implementation of web standards such as HTML, DOM, CSS, and others, entirely in JavaScript.

When we run JavaScript that manipulates the DOM in a browser, the browser provides the DOM. However, when we are running tests using a testing framework such as Jest in a Node.js environment, there isn't a DOM by default. This is where `jsdom` comes into play. `jsdom` provides a virtual DOM, thereby allowing our tests to run as though they were in a browser-like environment, even when they're running in Node.js.

Why do we need `jsdom`? In modern frontend development, particularly with frameworks such as React, Angular, and Vue, our JavaScript code often interacts directly with the DOM. For our tests to be useful, they need to be able to simulate this interaction. `jsdom` allows us to do this without needing to open a browser window.

In integration tests, we concentrate on the interactions between various modules. However, even if these interactions function as expected, there's still a possibility that the broader system may break. User journeys often involve multiple steps, so it's essential to have a process that seamlessly connects these steps to ensure the software continues to work reliably.

Learning about E2E tests using Cypress

E2E tests are at the top of the test pyramid. E2E tests simulate real user flows and interactions, testing the system as a whole. These tests help ensure that all parts of the application work together as expected, from the user interface to the backend systems.

We're going to use Cypress as the E2E test framework in this book. **Cypress** is a powerful tool for E2E testing of modern web applications. Its unique approach sets it apart from many other testing tools – instead of using Selenium, a common engine for many testing systems, Cypress operates directly on the actual browser, resulting in more reliable tests and a superior debugging experience.

Installing Cypress

You can either install Cypress into an existing project (like what we're doing in this book) or install it in another folder other than your project. Cypress has been added as a project dependency in the code base in GitHub (provided in the *Technical requirements* section), so you only need to run `npm install` in the project root (refer to the official documentation for more information: <https://docs.cypress.io/guides/getting-started/installing-cypress>).

Once you have installed the package, simply run `npx cypress open` to launch the configuration wizard:

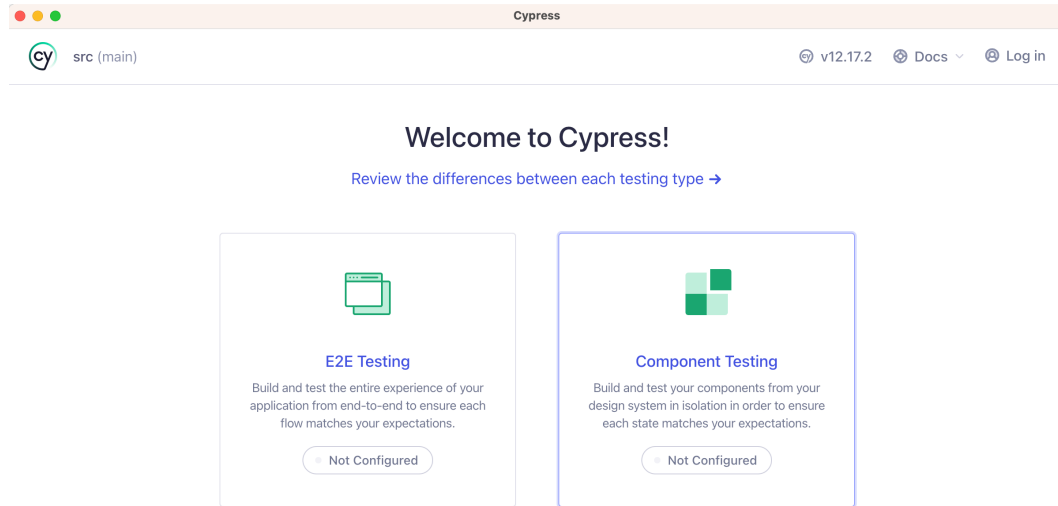


Figure 5.4: The Cypress wizard – choosing a test type

Follow the wizard to configure **E2E Testing**, and choose **Chrome** as the browser for running all the tests. After that, choose **Create new spec**:

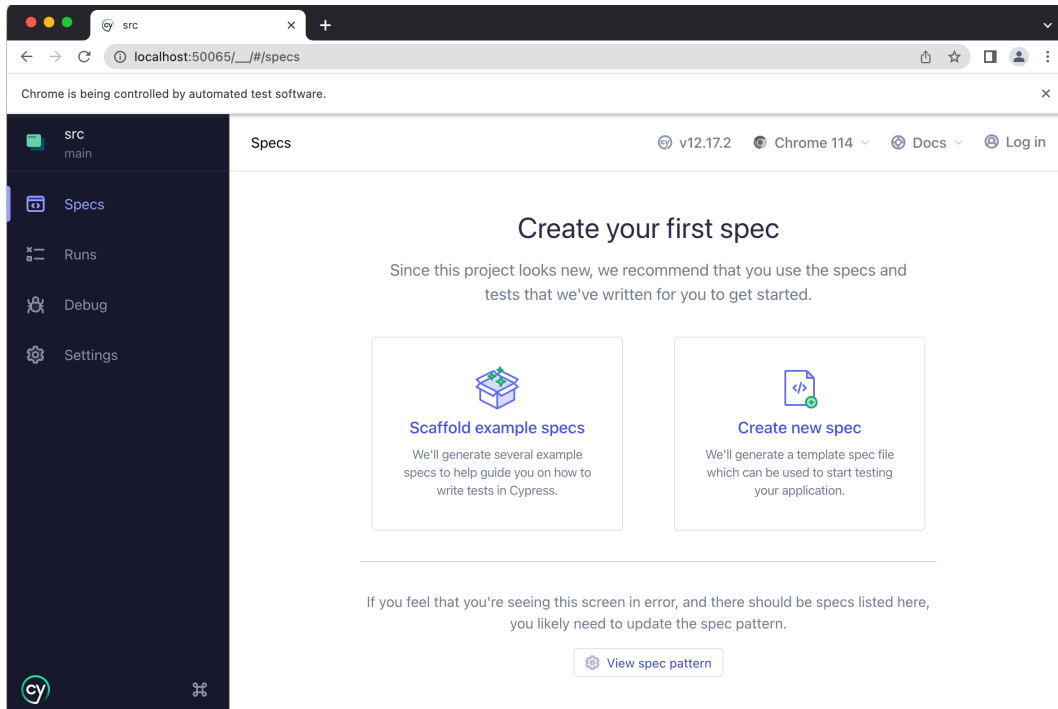


Figure 5.5: The Cypress wizard – creating a spec from a template

Cypress will create a folder with all the necessary files for us:

```
cypress
├─ downloads
├─ e2e
│   └─ quote-of-the-day.spec.cy.js
├─ fixtures
│   └─ example.json
└─ support
    ├── commands.js
    └─ e2e.js
```

Let's break his structure down:

- At the top level, we have the `cypress` directory, which is the root directory for all the Cypress-related files.
- The `downloads` directory is usually where files downloaded during Cypress tests would be stored; we're not going to use it here as we don't have anything to save at this stage.

- The `e2e` directory is where E2E test files are located. In this case, it contains `quote-of-the-day.spec.cy.js` (generated from the Cypress wizard), a Cypress test file for testing the quote-of-the-day feature of an application.
- The `fixtures` directory is a place to put external static data that your tests will use. We can put some static files that can be used in our tests (for example, if our tests need some JSON data for mocking the network's response).
- The `support` directory houses Cypress commands and support files, which we will not touch either.

Once we have the folder structure set up, we can proceed and write our first test.

Running our first E2E test

Let's modify the `quote-of-the-day.spec.cy.js` file to make it access a remote website. Cypress will actively watch the files under the `cypress/e2e/` folder, and whenever the content changes, it will rerun the test.

Make sure you have launched cypress in a terminal window (either Terminal for MacOS/Linux or Windows Terminal) with `npm run cypress:open`:

```
describe('quote of the day', () => {
  it('display the heading', () => {
    cy.visit('https://icodeit-juntao.github.io/quote-of-the-day/');
  })
});
```

In this code snippet, `describe` is used to declare a test suite – in this case, for the quote of the day feature. Within this suite, there's a single test case defined by it, labeled `display the heading`. The purpose of this test case is to visit a web page – in this instance, `https://icodeit-juntao.github.io/quote-of-the-day/` – and this web page returns a random quote each time the user refreshes the page.

However, it's important to note that this test case doesn't perform any actual tests or assertions yet. It merely navigates to the page. To make this a meaningful test, you would typically add assertions to check the state of specific elements on the page, such as the heading or a quote displayed:

```
it('display the heading', () => {
  cy.visit('https://icodeit-juntao.github.io/quote-of-the-day/');
  cy.contains("Quote of the day");
});
```

This code snippet is a meaningful test now. After visiting `https://icodeit-juntao.github.io/quote-of-the-day/`, this test now contains an additional check with the `cy.contains()` method. The `cy.contains()` method is used to search for and get a DOM element that contains the specified text – in this case, **Quote of the day**. This method will get the first element it finds that contains the text, and it will fail the test if no such element is found.

If the test can pass, we're confident that the URL is accessible to the public, and the page doesn't throw any exceptions:

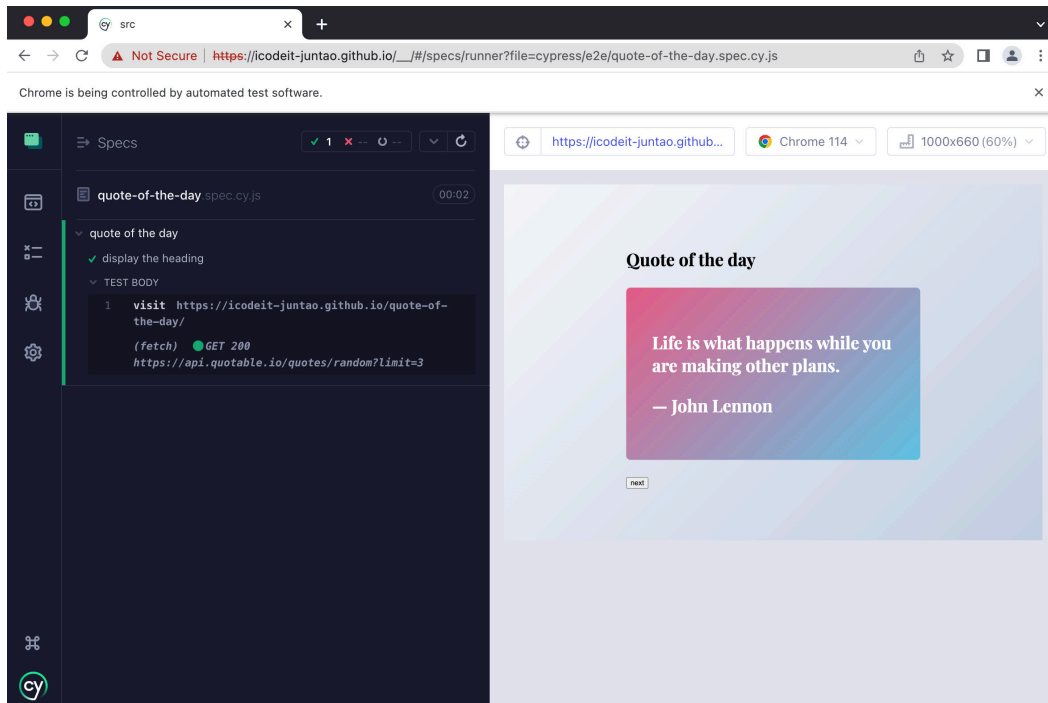


Figure 5.6: Running E2E tests inside Cypress Test Runner

Note that in *Figure 5.6*, on the right-hand side of the screen, you can see what is displayed on the real browser, while on the left-hand side, you can see the test cases and steps. You can even use the mouse to hover on a step to see the page snapshot at that point.

In addition, we can add another test case to verify that a quote container is present on the page; that is the most important part of the quote of the day application – to make sure a quote shows up:

```
it('display a quote', () => {
  cy.visit('https://icodeit-juntao.github.io/quote-of-the-day/');
  cy.get('[data-testid="quote-container"]').should('have.length', 1);
})
```

In this test, the `cy.get()` method is used to retrieve a DOM element by its `data-testid` attribute. This attribute is typically used for testing, allowing you to select elements without the need to worry about their CSS selectors or contents, which might change over time.

The element that's being selected in this test has a `data-testid` attribute of `quote-container`. Once the element is retrieved, the `should()` method is invoked to assert something about the state of that element. In this case, it checks that the length of the element (that is, the number of matching elements) is 1.

So, in this test, after navigating to the web page, it looks for an element with the `data-testid` attribute of `quote-container`, and checks that exactly one such element exists. If it does, the test will pass; if not (either because there are no matching elements or more than one), the test will fail.

That is awesome, but there is a problem here: what if the page isn't blank and the heading is rendered correctly, but the actual content of a quote doesn't show up for some reason? Alternatively, what if the quotes are visible but we're uncertain about the expected quotes when we're writing the tests? Consider, for instance, the <https://icodeit-juntao.github.io/quote-of-the-day/> website, which generates random quotes each time it's accessed. Different users may encounter different quotes at various times. To address these variables, we need a structured method for testing applications with such unpredictable behavior.

Intercepting the network request

In some cases, we don't want to send actual network requests for the UI to work, while in other cases, it's not practical to rely on the response directly. We want to verify whether the quote is rendered correctly by checking the content, but as the quoted content is generated randomly, we cannot predict it before we make the network request. This means we need a mechanism to pin down the response, but we would also like to send the request.

One way to achieve that is to intercept the network request that's sent to the endpoint and return some fixed data. In Cypress, we can do that through the `cy.intercept` API.

Firstly, we can define a data array in the `quote-of-the-day.spec.cy.js` file. It's a normal JavaScript array that contains the data we expect to return from the server side:

```
const quotes = [
  {
    content:
      "Any fool can write code that a computer can understand. Good
      programmers write code that humans can understand.",
    author: "Martin Fowler",
  },
  {
    content: "Truth can only be found in one place: the code.",
    author: "Robert C. Martin",
  },
]
```

```
    },  
    {  
      content:  
        "Optimism is an occupational hazard of programming: feedback is  
        the treatment.",  
      author: "Kent Beck",  
    },  
  ],  
];
```

In the testing code, we aim to capture any network requests that are sent to URLs beginning with `https://api.quotable.io/quotes/random`. Whenever the request is sent from React, Cypress will cancel the request and return the `quotes` array instead; this means the test doesn't depend on whether the remote service is working or not. That way, our test is more stable. We can see the code here:

```
it("display the quote content", () => {  
  cy.intercept("GET", "https://api.quotable.io/quotes/random*", {  
    statusCode: 200,  
    body: quotes,  
  });  
  
  cy.visit("https://icodeit-juntao.github.io/quote-of-the-day/");  
  
  cy.contains(  
    "Any fool can write code that a computer can understand. Good  
    programmers write code that humans can understand."  
  );  
  
  cy.contains(  
    "Martin Fowler"  
  );  
});
```

The `cy.intercept` function is being used to stub the HTTP GET request to the quote API. When such a request is detected, rather than letting the request go through to the actual API, Cypress will respond with a predefined HTTP response. This response has a status code of 200, indicating success, and the body of the response is set to be our predefined `quotes` data. This technique allows us to control the data being returned, making our test more deterministic and isolated from any potential instability or variation in the actual API.

The test then navigates to the quote web page. After the page is loaded, it verifies if the page contains the expected quote text and the quote author. If these two checks pass, the test case succeeds.